Documentación completa 2 2 Administración de usuarios

Backend City Care | Versión 1.0

Resultado. El módulo administrativo de usuarios está terminado en el backend. Permite crear, listar, buscar, consultar, editar, activar o desactivar cuentas y restablecer contraseñas. Todas las operaciones están protegidas por autenticación y rol administrador, cuentan con validación, contrato OpenAPI, cliente Orval y pruebas automatizadas.

Estado	Cobertura	Evidencia
COMPLETO	9 de 9 puntos del apartado 2.2	14 pruebas funcionales nuevas
VALIDADO	Autenticación 2.1 sin regresiones	23 de 23 pruebas aprobadas
SINCRONIZADO	OpenAPI y SDK Orval	6 operaciones administrativas generadas

1 Alcance terminado

Punto	Implementación	Estado
Crear usuario	Alta transaccional del perfil y su credencial	Listo
Validar rol y dependencia	Las referencias deben existir y el correo debe ser único	Listo
Transacción de alta	Si falla una escritura, no queda información parcial	Listo
Listar y buscar	Paginación, búsqueda y filtros combinables	Listo
Consultar detalle	Incluye perfil, rol, dependencia y estado	Listo
Editar usuario	Sincroniza el correo de users y credentials	Listo
Activar o desactivar	Baja lógica y revocación de sesiones	Listo
Restablecer contraseña	Hash nuevo, sesiones revocadas y evento auditable	Listo
Definir eliminación	No existe borrado físico; se usa is_active	Listo

2 Arquitectura del módulo

Las rutas administrativas pasan primero por el middleware de autenticación y después por el middleware de administrador. El controlador valida la solicitud con VineJS, aplica las reglas de negocio, ejecuta las escrituras relacionadas dentro de una transacción y finalmente transforma la respuesta pública.

Etapa	Responsabilidad	Archivo principal

Etapa	Responsabilidad	Archivo principal
1 Ruta	Expone el recurso bajo /api/v1/admin/users	start/routes.ts
2 Seguridad	Valida Bearer token, cuenta activa y rol admin	auth_middleware.ts y admin_middleware.ts
3 Validación	Revisa formato, unicidad y referencias	app/validators/user.ts
4 Negocio	Ejecuta consulta o modificación transaccional	admin_users_controller.ts
5 Persistencia	Usa users, credentials, roles, dependencies y tokens	Modelos Lucid
6 Respuesta	Oculta password_hash y expone datos públicos	user_transformer.ts

3 Catálogo de endpoints

Método	Ruta	Operación	Respuesta
GET	/api/v1/admin/users	Listar, buscar y filtrar	200
POST	/api/v1/admin/users	Crear usuario	201
GET	/api/v1/admin/users/{id}	Consultar detalle	200
PATCH	/api/v1/admin/users/{id}	Editar usuario	200
PATCH	/api/v1/admin/users/{id}/status	Activar o desactivar	200
PATCH	/api/v1/admin/users/{id}/password	Restablecer contraseña	200

3 1 Listado búsqueda y filtros

El listado devuelve data y meta. Los filtros son opcionales y se pueden combinar. El orden predeterminado es por id descendente. perPage admite como máximo 100 registros.

Parámetro	Tipo	Uso
page	number	Página solicitada; valor inicial 1
perPage	number	Elementos por página; valor inicial 20
search	string	Busca en nombre, apellido y correo
roleId	number	Filtra por rol
dependencyId	number	Filtra por dependencia
isActive	boolean	Filtra cuentas activas o inactivas

3 2 Alta de usuario

El alta recibe dependencia, rol, nombre, apellidos, correo, contraseña y campos opcionales. La contraseña se transforma a hash antes de la transacción. users y credentials se crean juntos y se revierten juntos si ocurre un error.

3 3 Edición de usuario

La edición es parcial. Debe enviarse al menos un campo. Si cambia el correo, el mismo valor se actualiza en users y credentials dentro de una sola transacción. Un administrador no puede quitarse a sí mismo el rol admin.

3 4 Baja lógica y reactivación

La cuenta no se elimina. El endpoint de estado modifica is_active. Cuando cambia a false, elimina todos los tokens de acceso del usuario dentro de la misma transacción. Un administrador no puede desactivar su propia cuenta.

3 5 Restablecimiento de contraseña

El administrador envía password y passwordConfirmation. El backend genera un hash seguro, reemplaza password_hash y revoca todas las sesiones del usuario. La contraseña nunca se devuelve en la respuesta ni se escribe en los registros de auditoría.

4 Seguridad y reglas de negocio

- Todas las rutas requieren Authorization: Bearer <token>.
- El usuario autenticado debe estar activo y tener un rol cuyo slug sea admin.
- La contraseña se almacena únicamente como hash en credentials.password_hash.
- La baja es lógica; no se expone un endpoint DELETE para usuarios.
- Desactivar una cuenta o restablecer su contraseña revoca todas sus sesiones.
- Se impide la autodesactivación y la pérdida accidental del propio rol administrador.
- Los eventos de creación, edición, estado y contraseña se registran con actorUserId y targetUserId.
- Los eventos de auditoría no incluyen contraseñas, hashes ni tokens.

5 Datos y transacciones

El apartado 2.2 no necesita tablas nuevas. Reutiliza la estructura creada para autenticación y roles. Por lo tanto, no se requiere ejecutar una migración adicional en MySQL para estas funciones.

Tabla	Uso en 2.2
users	Perfil, relación con rol y dependencia, y estado is_active
credentials	Correo de acceso y password_hash
roles	Autorización y asignación de rol
dependencies	Adscripción organizacional del usuario
auth_access_tokens	Revocación de sesiones al desactivar o restablecer contraseña

6 Formato de respuestas y errores

Las respuestas de usuario utilizan un transformador común. Incluyen datos públicos, rol y dependencia; nunca incluyen password_hash. El listado añade metadatos de paginación.

Código	Significado	Ejemplo de causa
200	Operación correcta	Consulta, edición, estado o contraseña
201	Recurso creado	Alta de usuario
401	No autenticado	Token ausente, inválido o revocado
403	Sin permiso	Usuario autenticado sin rol admin
404	No encontrado	El id de usuario no existe
409	Conflicto de integridad	Falta la credencial asociada
422	Solicitud inválida	Campos incorrectos o regla de autoprotección

Estructura uniforme de error:

```
{
  "error": {
    "code": "E_USER_NOT_FOUND",
    "message": "Usuario no encontrado"
  }
}
```

7 Contrato OpenAPI y cliente Orval

Swagger publica las rutas y esquemas del módulo. El archivo openapi/city-care.v1.yaml se sincronizó con el servidor y Orval generó sdk/generated/city-care.ts. El archivo generado no se edita manualmente.

Operation ID	Función generada	Ruta
listUsers	listUsers	GET /admin/users
createUser	createUser	POST /admin/users
getUser	getUser	GET /admin/users/{id}
updateUser	updateUser	PATCH /admin/users/{id}
updateUserStatus	updateUserStatus	PATCH /admin/users/{id}/status
resetUserPassword	resetUserPassword	PATCH /admin/users/{id}/password

Comando de regeneración:

```
npm run openapi:generate
```

8 Pruebas y verificación

La suite contiene 14 pruebas funcionales específicas del módulo 2.2. Junto con las 9 pruebas de autenticación, el resultado final es 23 de 23 pruebas aprobadas.

Grupo	Casos comprobados
Autorización	Sin token 401 y sin rol admin 403
Consultas	Listado, búsqueda, filtros, paginación, detalle y 404
Alta	Usuario y credencial creados en conjunto
Edición	Perfil, rol, dependencia y sincronización del correo
Autoprotección	No quitar propio rol ni desactivar propia cuenta
Estado	Desactivar, reactivar y revocar sesiones
Contraseña	Nuevo hash, revocación y login con contraseña nueva
Eliminación	La ruta DELETE no existe

Comandos de cierre ejecutados correctamente:

```
npm run typecheck
npm run lint
npm test
npm run build
node ace list:routes
```

9 Guía de operación

Paso	Acción
1	Iniciar MySQL y confirmar que la base city-care está disponible
2	Ejecutar npm run dev en el backend
3	Iniciar sesión en POST /api/v1/auth/login con una cuenta admin activa
4	Copiar el token y enviar Authorization: Bearer <token>
5	Usar las rutas /api/v1/admin/users desde Swagger o el SDK Orval
6	Si cambia el contrato, sincronizar Swagger y volver a ejecutar Orval

10 Criterio de cierre del apartado 2.2

El apartado 2.2 se considera terminado porque las nueve capacidades definidas están implementadas, protegidas, documentadas y comprobadas automáticamente. No queda una migración pendiente. La conexión de estas funciones con la vista administrativa del frontend corresponde a una etapa posterior y no cambia el estado de cierre del backend 2.2.